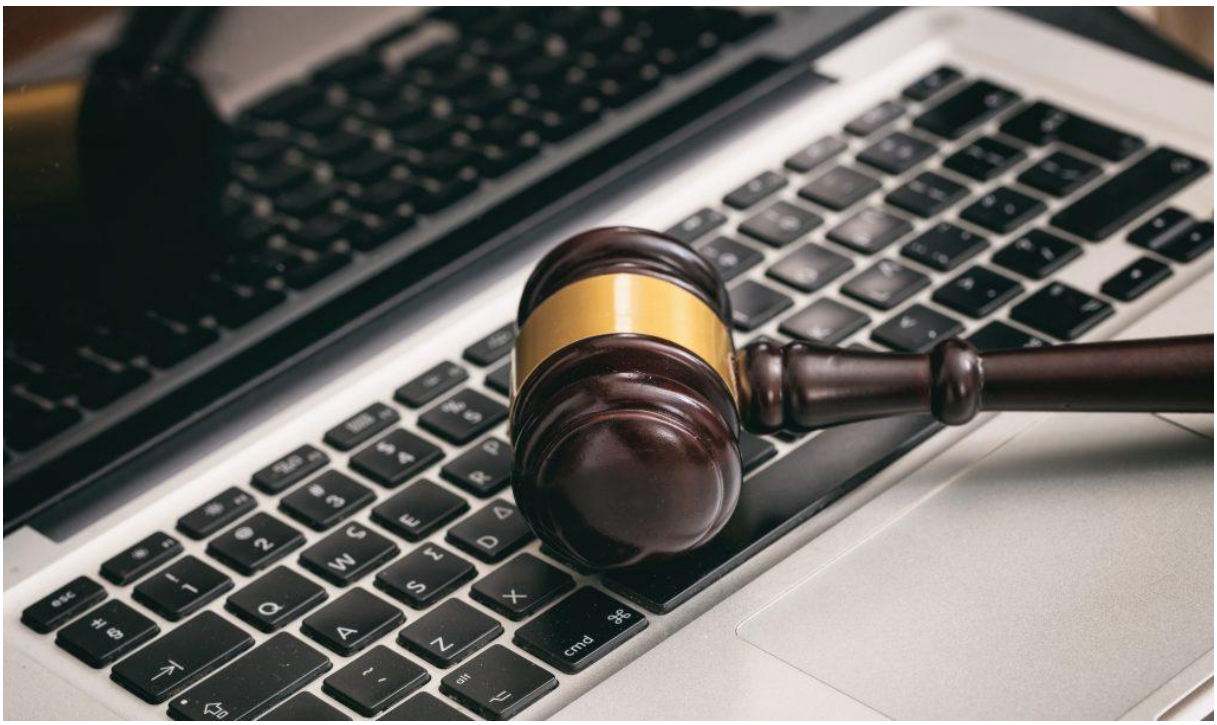
Loser wins!

A Problem-Based Learning (PBL) project for students in the Advanced Algorithms 3 module



Henri HAGBE

Background

You are joining an ambitious young startup that wants to disrupt online auctions: LowBid. Your team is preparing the launch of a platform designed to turn a premium item into a viral event. The concept—simple on the surface—is called “Lowest Unique Bid Wins”.

Here, the auction is inverted: the winner is not the highest bidder, but the person who proposes the lowest price that is proposed by exactly one person (integer bid ≥ 0). This creates immediate strategic tension: you must dare to go low… without going so low that you land where everyone else clusters.

From the seller’s perspective, the model is formidable: every attempt generates revenue, because every bid has a cost. The business value does not come only from the final price, but from the volume of bids (a “game” effect): the more participants there are and the more they try, the higher the revenue.

To prevent everyone from rushing to 0 and a few low prices (which would create constant ties and kill the appeal), the startup introduces a risk premium: the lower the bid, the more expensive the bid cost. A reference formula is:

$$bid_cost(price) = base_cost + \alpha/(price+1)$$

It penalizes “low bidders” who spam very small prices, while preserving genuine strategic depth.

α (alpha) is the parameter that controls the intensity of the risk premium. The larger α is, the more expensive low bids become (so we strongly discourage spamming 0, 1, 2…).

The smaller α is, the weaker the risk premium becomes (the game moves closer to a “classic” auction where low bids cost almost the same as higher bids).

Your mission looks like that of a tech team that wants to “go big”: deliver a reliable engine quickly, able to handle thousands of bids, produce explainable outcomes, and support a convincing demo for investors to raise millions of euros. You should therefore prioritize a solid, testable, and justifiable design.

Before you start, note that this problem situation is intentionally open-ended: you will have to make choices (data structures, code organization, rules for edge cases) and be able to justify them.

Work to be done

You will work as a team over one PBL cycle. The goal is to develop a complete, testable program, and to be able to explain and justify your technical choices during the restitution session.

1. Load data:

    - Load a bid dataset (list of pairs {player, price}) from a text file (simple CSV) or generate a demo dataset.
        - Build the BST of prices: insert bids, handling multiple players for the same price (a list attached to each node).
        - Display the auction state using an in-order traversal (sorted prices) to validate the structure and bid distribution.

    - Compute the costs paid by players using the risk premium, then compute the seller's revenue (sum of costs).

2. Query / search:
    - Find the lowest unique price and determine the winner (price, player).
        - Handle the case where no price is unique (no winner): define and apply a rule (e.g., round cancelled, or no winner).
        - Support analysis queries: winning price, price distribution, total number of bids, average cost per player.
    - Implement a multi-round simulation (at least 500 rounds) with at least two player strategies.
        - Compare strategies: win rate, average profit per round, average seller revenue, and the effect of parameters (base_cost, α).
        - Discuss limitations: BST degeneration, alternatives (dictionary + sort, heap), and performance impact.
        - The price that follows a given price in the BST (successor) and why it matters (e.g., find the next candidate after a non-unique price).
        - The price that precedes a given price in the BST (predecessor) and why it matters (e.g., navigation in price order).
        - Let humans play at the same time as the computer and reflect on what makes a winning strategy.

🎬 Kick-off session — Discover and frame the problem

Goal:

- Check understanding of the problem
- Establish the structural/data-structure stakes

🧠 Kick-off session — Reflexion on game theory

- Why is bid 0 not a dominant strategy with the risk premium?
- Is the system zero-sum? Justify your answer.
- What information is necessary to determine the winner?
- Can you determine the winner without sorting prices?

🎬 Intermediate session — Framing questions (before the team practical)

- Are there data structures that can keep prices dynamically ordered?
- What is the difference between sorting at the end and maintaining order continuously?
- If bids arrive as a continuous stream, which solution is the most appropriate?

⚙️ Algorithms and complexity

- What is the insertion complexity in:
  - A sorted array?
  - A balanced BST?
  - A dictionary + sorting at the end?
- In which cases can a BST become inefficient?
- How can degeneration be avoided?

🎲 Strategy

- In your model, if all players choose randomly, what happens?
- If everyone bids 0?
- Is there a “stable” price toward which the system converges?

Resources to address the problem situation

- Lecture 1 and tutorial 1 « Binary Search Trees ».
- Lecture 1 and tutorial 1 « General Trees ».
- Course of the first semester of first year « Python Language »
- Optionnal : « applications with IHM »

Expected deliverables

- A functional Python script meeting the requirements above.
- 📦 Technical constraints
  - Implement a Binary Search Tree (BST).
  - Handle multiple players for the same key.
  - Provide:
    i. Insertion, in-order traversal, lowest-unique search, conditional deletion, and multi-round simulation.
  - Produce a reasoned analysis.
  - Prohibitions:
    i. No Python sorting to solve the core problem.
- A short report explaining:
  - The project structure.
  - Technical choices: data structures, algorithms used, ...
  - Difficulties encountered and how they were resolved.

Particular attention will be paid to good programming practices and to team organization throughout the PBL cycle.

Learning objectives for the PBL (APP)

- Implement a complete BST (insertion, traversal, search) and justify how it works.
- Analyze complexity on average and in the worst case, and identify degeneration scenarios.
- Simulate multiple rounds with player strategies and extract relevant statistics.
- Produce an argued economic and strategic analysis (seller profitability, player rationality, effect of the risk premium).